

AWS CERTIFIED

AI Practitioner

AIF-C01

Guía de estudio completa

Basada en el exam guide oficial de AWS
docs.aws.amazon.com/aws-certification/latest/ai-practitioner-01/

AWS Certified AI Practitioner (AIF-C01)

Guía de estudio completa para el examen AWS Certified AI Practitioner.

Basada en el exam guide oficial de AWS (docs.aws.amazon.com/aws-certification/latest/ai-practitioner-01/).

Datos del examen

- **50 preguntas puntuadas** + 15 sin puntuar (no identificables, no afectan el resultado).
- **Puntaje:** escala 100–1000, mínimo para aprobar **700**.
- **Sin penalización** por adivinar; pregunta sin responder cuenta como incorrecta.
- **Tipos de pregunta:** opción múltiple (1 correcta, 3 distractores), respuesta múltiple (2+ correctas de 5+), ordenamiento (ordenar 3-5 pasos), matching (emparejar hasta 7 pares).
- **Modelo de scoring compensatorio:** no hace falta aprobar cada dominio, solo el puntaje total.
- **Candidato objetivo:** hasta 6 meses de exposición a AI/ML en AWS. Usa soluciones, no necesariamente las construye.
- **Fuera de alcance:** programar/entrenar modelos, feature engineering, hyperparameter tuning, matemática/estadística de ML, construir pipelines, implementar seguridad o governance en profundidad. El examen es sobre *entender y elegir*, no *implementar*.

Dominios y ponderación

Dominio	% del examen
1. Fundamentos de AI y ML	20%
2. Fundamentos de GenAI	24%
3. Aplicaciones de Foundation Models	28%
4. Guías de AI Responsable	14%
5. Seguridad, Compliance y Governance para soluciones de AI	14%

Los dominios 2 y 3 (GenAI + Foundation Models) son más de la mitad del examen: ahí conviene poner el foco de estudio.

Dominio 1: Fundamentos de AI y ML (20%)

1.1 Conceptos básicos y terminología

- **AI vs ML vs Deep Learning vs GenAI vs Agentic AI:** AI es el campo general; ML es un subconjunto que aprende de datos en vez de reglas explícitas; Deep Learning usa redes neuronales de varias capas; GenAI genera contenido nuevo; Agentic AI usa modelos para razonar y actuar de forma autónoma con herramientas.
- **Entrenamiento vs inferencia:** entrenar es ajustar los parámetros del modelo con datos; inferir es usar el modelo ya entrenado para predecir sobre datos nuevos.
- **Tipos de inferencia:** batch (procesa lotes grandes, no en tiempo real), real-time (respuesta inmediata), asíncrona (se procesa en background, se notifica al terminar), serverless (sin gestionar infraestructura).
- **Tipos de datos:** etiquetados (labeled, tienen la respuesta correcta) vs no etiquetados (unlabeled); tabulares, series de tiempo, imagen, texto; estructurados vs no estructurados.
- **Tipos de aprendizaje:**
 - **Supervisado:** entrena con datos etiquetados (ej. clasificación, regresión).
 - **No supervisado:** encuentra patrones sin etiquetas (ej. clustering).
 - **Por refuerzo:** aprende por prueba y error con un sistema de recompensas.
- **Bias y fairness:** bias es cuando el modelo favorece sistemáticamente ciertos resultados por sesgos en los datos de entrenamiento; fairness es diseñar para que el modelo trate equitativamente a distintos grupos.

1.2 Casos de uso prácticos

- **Cuándo usar AI/ML:** para asistir decisiones humanas, escalar soluciones, automatizar tareas repetitivas.
- **Cuándo NO usarlo:** cuando el costo-beneficio no cierra, o cuando se necesita un resultado exacto y determinístico en vez de una predicción (ej. cálculo de impuestos).
- **Técnicas según el problema:** regresión (predecir un valor numérico), clasificación (predecir una categoría), clustering (agrupar sin etiquetas previas).
- **Aplicaciones reales:** visión por computadora, NLP, reconocimiento de voz, sistemas de recomendación, detección de fraude, forecasting, knowledge bases, agentic AI.
- **Servicios administrados de AWS relevantes:**
 - **Amazon SageMaker AI:** plataforma completa para construir, entrenar y desplegar modelos propios.
 - **Amazon Transcribe:** voz a texto.
 - **Amazon Translate:** traducción de idiomas.
 - **Amazon Comprehend:** NLP (sentimiento, entidades, idioma).
 - **Amazon Lex:** chatbots conversacionales.
 - **Amazon Polly:** texto a voz.

- **Modelo tradicional vs foundation model:** usar modelo tradicional cuando hay requisitos regulatorios estrictos, se necesita explicabilidad total, o el caso es acotado y específico; usar FM cuando la tarea es general, hay poco tiempo/dato para entrenar desde cero, o se necesita lenguaje natural.

1.3 Ciclo de vida de desarrollo de AI/ML

- **Pipeline típico:** recolección de datos → preparación → entrenamiento → evaluación → despliegue → monitoreo → reentrenamiento.
- **Origen de foundation models:** modelos pre-entrenados open source, o entrenar modelos propios desde cero (mucho más caro y lento).
- **Poner un modelo en producción:** servicio de API administrado (ej. Amazon Bedrock) vs self-hosted API (vos gestionás la infraestructura, ej. SageMaker endpoints).
- **Servicios AWS por etapa del pipeline:** Amazon Bedrock, Amazon Quick, Kiro, SageMaker AI.
- **MLOps:** experimentación, procesos repetibles, sistemas escalables, gestión de deuda técnica, production readiness, monitoreo y reentrenamiento de modelos.
- **Métricas de performance del modelo:** accuracy, precision, recall, F1 score.
- **Métricas de negocio:** costo por usuario, costos de desarrollo, feedback de clientes, ROI.

Dominio 2: Fundamentos de GenAI (24%)

2.1 Conceptos básicos de GenAI

- **Tokens:** unidades mínimas de texto que procesa el modelo (palabras, subpalabras o caracteres).
- **Chunking:** dividir documentos grandes en fragmentos manejables antes de indexarlos o pasarlos a un modelo.
- **Embeddings:** representación numérica (vector) del significado de un texto, imagen u otro dato, usada para medir similitud semántica.
- **Prompt engineering:** diseñar la entrada al modelo para obtener mejores resultados.
- **LLM / FM / modelos multimodales / modelos de difusión:** LLM procesa/genera texto; FM (foundation model) es un modelo grande pre-entrenado adaptable a muchas tareas; multimodal maneja más de un tipo de dato (texto+imagen); difusión genera imágenes partiendo de ruido.
- **Casos de uso de GenAI:** generación de imagen/video/audio, resúmenes, asistentes de AI, traducción, generación de código, agentes de atención al cliente, búsqueda, motores de recomendación.
- **Ciclo de vida de un FM:** selección de datos → selección de modelo → pre-entrenamiento → fine-tuning → evaluación → despliegue → feedback.
- **Pricing basado en tokens:** se cobra por tokens de entrada y salida; afecta directamente el costo y en algunos casos la latencia.
- **Context engineering:** diseñar y gestionar todo el contexto (no solo el prompt) que recibe el modelo: historial, documentos recuperados, instrucciones del sistema.

- **Conceptos de agentic AI:** patrones multi-agente, **Model Context Protocol (MCP)** para conectar agentes a sistemas externos, comunicación entre agentes, gestión de memoria, uso de herramientas (tools), orquestación de workflows.

2.2 Capacidades y límites de GenAI

- **Ventajas:** adaptabilidad, capacidad de respuesta, conversación natural, generación de contenido.
- **Desventajas:** alucinaciones (inventa información con confianza), baja interpretabilidad, inexactitud, no determinismo (misma entrada puede dar salidas distintas).
- **Factores para elegir un modelo:** tipo de modelo, requisitos de performance, capacidades, restricciones, compliance, costo, latencia, complejidad.
- **Métricas de valor de negocio:** performance cross-domain, ROI, eficiencia, conversion rate, ARPU, accuracy, customer lifetime value.

2.3 Infraestructura y tecnologías de AWS para GenAI

- **Servicios clave:** Amazon Bedrock (acceso a FMs vía API administrada), Amazon SageMaker AI y SageMaker JumpStart (entrenar/personalizar modelos), Amazon Quick, Kiro, Strands Agents, Amazon Bedrock AgentCore (infraestructura para agentes).
- **Ventajas de usar servicios de AWS:** accesibilidad, barrera de entrada baja, eficiencia, costo-efectividad, velocidad de salida al mercado.
- **Beneficios de la infraestructura AWS:** seguridad, compliance, responsabilidad, safety.
- **Cost tradeoffs:** responsiveness, disponibilidad, redundancia, performance, cobertura regional, pricing por token, **provisioned throughput** (capacidad reservada, más cara pero predecible) vs on-demand, modelos personalizados (más costo de entrenamiento pero mejor ajuste).

Dominio 3: Aplicaciones de Foundation Models (28% — el dominio más grande)

3.1 Consideraciones de diseño para apps con FMs

- **Criterios de selección de un FM:** costo, modalidad, latencia, soporte multilingüe, tamaño del modelo, complejidad, customización, largo de input/output, prompt caching.
- **Parámetros de inferencia:**
 - **Temperature:** controla aleatoriedad; baja = más determinístico/conservador, alta = más creativo/variado.
 - **Largo de input/output:** afecta costo y latencia.
- **RAG (Retrieval Augmented Generation):** combina un FM con una base de conocimiento externa recuperada en tiempo de consulta, para dar respuestas más precisas y actualizadas sin reentrenar el modelo. En AWS: **Amazon Bedrock Knowledge Bases**.

- **Bases de datos vectoriales en AWS:** Amazon OpenSearch Service, Amazon Aurora, Amazon Neptune, Amazon RDS for PostgreSQL (con pgvector).
- **Tradeoffs de costo entre formas de customización de un FM** (de menor a mayor costo/esfuerzo):
 1. **In-context learning:** dar ejemplos directo en el prompt, sin tocar el modelo.
 2. **RAG:** sumar contexto externo recuperado, sin reentrenar.
 3. **Fine-tuning:** ajustar pesos del modelo con datos propios.
 4. **Pre-training:** entrenar desde cero, el más caro y lento.
 5. **Model distillation:** entrenar un modelo chico para imitar a uno grande, reduce costo de inferencia.
- **AI Agents:** sistemas que usan un FM para razonar, planificar y ejecutar acciones (llamar herramientas, APIs) de forma autónoma para cumplir un objetivo.

3.2 Técnicas de prompt engineering

- **Conceptos:** contexto, instrucción, negative prompts (indicar qué NO debe hacer/incluir el modelo).
- **Técnicas:**
 - **Zero-shot:** sin ejemplos, solo la instrucción.
 - **Single-shot / few-shot:** con uno o varios ejemplos en el prompt.
 - **Chain-of-thought:** pedirle al modelo que razone paso a paso antes de responder.
 - **Prompt templates:** plantillas reutilizables con variables.
- **Buenas prácticas:** mejorar calidad de respuesta, experimentar, usar guardrails, ser específico y conciso.
- **Riesgos del prompt engineering:**
 - **Prompt exposure:** filtración del prompt/system prompt.
 - **Prompt poisoning:** inyectar contenido malicioso en los datos de contexto.
 - **Prompt hijacking:** el atacante redirige el propósito del prompt.
 - **Jailbreaking:** eludir las restricciones de seguridad del modelo.
- **Amazon Bedrock Prompt Management:** versionado y gestión centralizada de prompts.

3.3 Entrenamiento y fine-tuning de FMs

- **Pre-training:** entrenar el modelo base desde cero con datos masivos.
- **Fine-tuning:** ajustar un modelo ya pre-entrenado con datos específicos de un dominio.
- **Continuous pre-training:** seguir entrenando el modelo base con más datos generales (sin ser tarea específica).
- **Distillation:** transferir el conocimiento de un modelo grande a uno chico.
- **Métodos de fine-tuning:** instruction tuning (enseñar a seguir instrucciones), adaptación a dominios específicos, transfer learning, continuous pre-training.
- **Preparar datos para fine-tuning:** curación, governance, tamaño del dataset, etiquetado, representatividad, **RLHF** (reinforcement learning from human feedback: usar feedback humano para ajustar el comportamiento del modelo).

3.4 Evaluación de performance de FMs

- **Enfoques de evaluación:** human-in-the-loop, benchmark datasets, **Amazon Bedrock Model Evaluation**.
- **Métricas específicas de FMs:**
 - **ROUGE:** mide calidad de resúmenes (overlap con un resumen de referencia).
 - **BLEU:** mide calidad de traducción automática.
 - **BERTScore:** similitud semántica usando embeddings.
 - **LLM-as-a-judge:** usar otro LLM para evaluar las respuestas del modelo.
- **Evaluar si el FM cumple el objetivo de negocio:** productividad, engagement de usuarios, task engineering.
- **Evaluar aplicaciones completas** (no solo el modelo): RAG, agentes, workflows.
- **Métricas de alineación con objetivos de negocio:** task completion rate, satisfacción del usuario, costo por interacción.

Dominio 4: Guías de AI Responsable (14%)

4.1 Desarrollo responsable de sistemas de AI

- **Características de AI responsable:** bias, fairness, inclusividad, robustez, safety, veracidad.
- **Herramientas para identificarlas:** **Amazon Bedrock Guardrails** (filtra contenido dañino, PII, temas prohibidos, detecta alucinaciones).
- **Prácticas responsables al elegir un modelo:** consideraciones ambientales, sustentabilidad (modelos más grandes consumen más energía).
- **Riesgos legales de GenAI:** reclamos por infracción de propiedad intelectual, outputs sesgados, pérdida de confianza del cliente, riesgo para el usuario final, alucinaciones.
- **Características de datasets responsables:** inclusividad, diversidad, fuentes curadas, datasets balanceados.
- **Bias vs variance:** bias afecta desproporcionadamente a grupos demográficos e introduce inexactitud sistemática; variance se relaciona con overfitting (memoriza el dato de entrenamiento, generaliza mal) vs underfitting (el modelo es demasiado simple para capturar el patrón).
- **Herramientas para detectar/monitorear bias y veracidad:** análisis de calidad de etiquetas, auditorías humanas, análisis por subgrupos.

4.2 Modelos transparentes y explicables

- **Transparente/explicable vs no:** un modelo explicable permite entender por qué llegó a una decisión (ej. árboles de decisión, regresión lineal); uno no explicable es una "caja negra" (ej. redes neuronales profundas, muchos FMs).
- **Herramientas para identificar modelos transparentes:** **Amazon SageMaker Model Cards** (documentan propósito, datos y limitaciones del modelo), **Amazon Bedrock Model Evaluations**,

modelos/datos/licencias open source.

- **Tradeoff entre safety y transparencia:** medir interpretabilidad vs performance — modelos más complejos suelen ser más precisos pero menos explicables.
- **Diseño centrado en el humano para AI explicable:** mecanismos de feedback del usuario, transparencia en la decisión de la AI.

Dominio 5: Seguridad, Compliance y Governance para soluciones de AI (14%)

5.1 Métodos para asegurar sistemas de AI

- **Servicios/features de AWS para seguridad de AI:**
 - **IAM:** roles, políticas y permisos para controlar acceso.
 - **Encriptación:** en tránsito y en reposo.
 - **Amazon Macie:** detecta datos sensibles (PII) en S3.
 - **AWS PrivateLink:** conectividad privada sin pasar por internet público.
 - **Shared Responsibility Model:** aplica también a AI — AWS asegura la infraestructura del servicio, vos aseguras tus datos y configuración.
 - **Amazon Bedrock AgentCore Identity y Policy en AgentCore:** gestión de identidad y políticas para agentes.
 - **Amazon Bedrock Guardrails:** control de contenido y comportamiento del modelo.
- **Source citation y trazabilidad de datos:** data lineage, data cataloging, **Amazon SageMaker Model Cards**.
- **Buenas prácticas de ingeniería de datos segura:** evaluar calidad de datos, tecnologías que mejoran la privacidad, control de acceso a datos, integridad de datos.
- **Consideraciones de seguridad y privacidad para AI:** seguridad de la aplicación, detección de amenazas, gestión de vulnerabilidades, protección de infraestructura, **prompt injection**, encriptación en reposo/tránsito, prevención de fuga de datos, filtrado y validación de outputs, auditoría y logging de interacciones con AI, toxicidad.
- **Detección de alucinaciones y grounding:** RAG grounding (anclar respuestas a fuentes recuperadas), validación de outputs, confidence scoring.

5.2 Governance y compliance regulatorio

- **Servicios de AWS para governance y compliance:** AWS Config, Amazon Inspector, AWS Artifact, AWS CloudTrail, AWS Trusted Advisor.
- **Estrategias de data governance:** ciclos de vida de datos, logging, residencia de datos (data residency), monitoreo, observación, retención.
- **Procesos de governance:** políticas, cadencia de revisión, estrategias de revisión, frameworks de governance (por ejemplo el **Generative AI Security Scoping Matrix** de AWS), estándares de

transparencia, requisitos de entrenamiento del equipo.

Servicios AWS dentro del alcance del examen

Lista oficial (no exhaustiva) de servicios que pueden aparecer en preguntas.

Categoría	Servicios
Analytics	AWS Data Exchange, Amazon EMR, AWS Glue, AWS Glue DataBrew, AWS Lake Formation, Amazon OpenSearch Service, Amazon Quick, Amazon Redshift
Cloud Financial Management	AWS Budgets, AWS Cost Explorer
Compute	Amazon EC2, AWS Lambda
Containers	Amazon ECS, Amazon EKS
Database	Amazon Aurora, Amazon DocumentDB, Amazon DynamoDB, Amazon ElastiCache, Amazon Neptune, Amazon RDS
Developer Tools	Kiro, Strands Agents
Machine Learning	Amazon Bedrock, Amazon Bedrock AgentCore, Amazon Comprehend, Amazon Lex, Amazon Nova, Amazon Personalize, Amazon Polly, Amazon Rekognition, Amazon SageMaker AI, Amazon SageMaker JumpStart, Amazon Textract, Amazon Transcribe, Amazon Translate, AWS Transform
Management and Governance	AWS CloudTrail, Amazon CloudWatch, AWS Config, AWS Trusted Advisor, AWS Well-Architected Tool
Networking and Content Delivery	Amazon CloudFront, Amazon VPC
Security, Identity, and Compliance	AWS Artifact, IAM, Amazon Inspector, AWS KMS, Amazon Macie, AWS Secrets Manager
Storage	Amazon S3, Amazon S3 Glacier

Foco especial: Amazon Bedrock (y sus componentes: Knowledge Bases, Guardrails, Prompt Management, AgentCore, Model Evaluation) es el servicio más preguntado del examen. Repasarlo a fondo.

Plan de repaso sugerido

1. **Servicios de ML de AWS** (Comprehend, Lex, Polly, Transcribe, Translate, Rekognition, Textract, Personalize): qué hace cada uno en una frase, sin entrar en detalle técnico.

2. **Amazon Bedrock a fondo:** qué es, Knowledge Bases (RAG), Guardrails, Agents, Model Evaluation, Prompt Management, pricing por token vs provisioned throughput.
3. **Conceptos de GenAI:** tokens, embeddings, chunking, prompt engineering (técnicas y riesgos), parámetros de inferencia (temperature).
4. **RAG y customización de modelos:** diferencia entre in-context learning, RAG, fine-tuning, pre-training y distillation — y su orden de costo/esfuerzo.
5. **Responsible AI:** bias/fairness, Guardrails, Model Cards, explicabilidad.
6. **Seguridad y governance:** shared responsibility aplicado a AI, servicios de compliance (Artifact, Config, CloudTrail, Trusted Advisor, Inspector), prompt injection y sus mitigaciones.
7. **Practicar con preguntas de escenario:** el examen mezcla conceptos, no pregunta definiciones sueltas.

Errores comunes a evitar

- Confundir **fine-tuning** con **RAG**: fine-tuning cambia los pesos del modelo; RAG solo agrega contexto externo en el momento de la consulta, sin tocar el modelo.
- Pensar que GenAI siempre es la mejor opción: para resultados exactos y determinísticos, un modelo tradicional o una regla de negocio puede ser mejor.
- Confundir **AWS CAF** y **Well-Architected** (del CLF-C02) con los frameworks específicos de AI responsable de este examen — son marcos distintos.
- Asumir que Bedrock Guardrails elimina las alucinaciones: las reduce/filtra, no las elimina.
- No diferenciar model card (documentación del modelo) de model evaluation (medir performance).